



МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Дальневосточный федеральный университет»

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ
Кафедра математического и компьютерного моделирования

Кулахсзян Сергей Грайрович

ВЫБОРКА

ЛАБОРАТОРНАЯ РАБОТА № 1

Направление подготовки 02.03.01 сэт Сквозные цифровые технологии,
бакалавриатская программа «Математика и компьютерные науки»
Очной формы обучения

г. Владивосток
2026

Содержание

1	Генерация выборок	2
2	Реализация выборочных характеристик	2
3	Сравнение расчетных характеристик	3
4	Построение графиков	3
4.1	Равномерное распределение	4
4.2	Распределение Бернулли	5
4.3	Биномиальное распределение	6
4.4	Нормальное распределение	8

1 Генерация выборок

Равномерное распределение $X \sim U(a = -1, b = 4)$:

```
1 data["uniform"][100]["X"] = stats.uniform.rvs(loc=-1, scale=5,  
2 data["uniform"][1000]["X"] = stats.uniform.rvs(loc=-1,  
scale=5, size=1000, random_state=random_state)
```

Распределение Бернулли $X \sim \text{Bernoulli}(p = 0.25)$:

```
1 data["bernoulli"][100]["X"] = stats.bernoulli.rvs(p=0.25,  
size=100, random_state=random_state)  
2 data["bernoulli"][1000]["X"] = stats.bernoulli.rvs(p=0.25,  
size=1000, random_state=random_state)
```

Биномиальное распределение $X \sim \text{Bin}(n = 17, p = 0.25)$:

```
1 data["binom"][100]["X"] = stats.binom.rvs(p=0.25, n=17,  
size=100, random_state=random_state)  
2 data["binom"][1000]["X"] = stats.binom.rvs(p=0.25, n=17,  
size=1000, random_state=random_state)
```

Нормальное распределение $X \sim N(M = 15, \sigma = 5)$:

```
1 data["norm"][100]["X"] = stats.norm.rvs(loc=15, scale=5,  
size=100, random_state=random_state)  
2 data["norm"][1000]["X"] = stats.norm.rvs(loc=15, scale=5,  
size=1000, random_state=random_state)
```

2 Реализация выборочных характеристик

Выборочное среднее:

$$\bar{x}_B = \sum_{i=1}^n \frac{x_i}{n}$$

```

1 def mean(x: list[float]) -> float:
2     return sum(x) / len(x)

```

Выборочная дисперсия:

$$D_B = \sum_{i=1}^n \frac{(x_i - \bar{x}_B)^2}{n}$$

```

1 def dispersion(x: list[float]) -> float:
2     x_mean = mean(x)
3     return sum((x_i - x_mean) ** 2 for x_i in x) / len(x)

```

Выборочное стандартное отклонение:

$$\sigma_B = \sqrt{D_B}$$

```

1 def standard_deviation(x: list[float]) -> float:
2     return dispersion(x) ** 0.5

```

3 Сравнение расчетных характеристик

№ выборки (распределение-размер)	Среднее		Дисперсия		Ст. отклонение	
	моё	numpy	моя	numpy	моё	numpy
№ 1 (равномерное-100)	1.4937	1.4937	2.1570	2.1570	1.4687	1.4687
№ 2 (равномерное-1000)	1.4820	1.4820	2.0780	2.0780	1.4415	1.4415
№ 3 (Бернулли-100)	0.2700	0.2700	0.1971	0.1971	0.4440	0.4440
№ 4 (Бернулли-1000)	0.2440	0.2440	0.1845	0.1845	0.4295	0.4295
№ 5 (биномиальное-100)	4.2600	4.2600	3.6124	3.6124	1.9006	1.9006
№ 6 (биномиальное-1000)	4.2290	4.2290	3.2526	3.2526	1.8035	1.8035
№ 7 (нормальное-100)	15.9197	15.9197	30.4590	30.4590	5.5190	5.5190
№ 8 (нормальное-1000)	14.9599	14.9599	26.9451	26.9451	5.1909	5.1909

4 Построение графиков

Для начало определяется длина частотного интервала:

$$h = \frac{R}{k}$$

$$R = X_{\max} - X_{\min}$$

$$k = 1 + 3.322 \lg n$$

Тут R – размах, k – число частотных интервалов, h – длина частотного интервала

Для каждого частотного интервала $[X_{\min} + ih; X_{\min} + (i + 1)h], i = \overline{0, k}$ определяется значение относительной частоты:

$$w_i = \frac{m_i}{n},$$

где m_i – частота вхождений элементов выборки в частотный интервал $[X_{\min} + ih; X_{\min} + (i + 1)h]$, n – объём выборки.

Плотность относительной частоты тогда будет равна:

$$h_i = \frac{w_i}{h}$$

Таким образом в гистограмме для каждого частотного интервала будет определён h_i

4.1 Равномерное распределение

Плотность распределений для равномерного распределения имеет вид:

$$f_X(x) = \frac{1}{b - a}$$

Код построения графиков будет иметь вид:

```

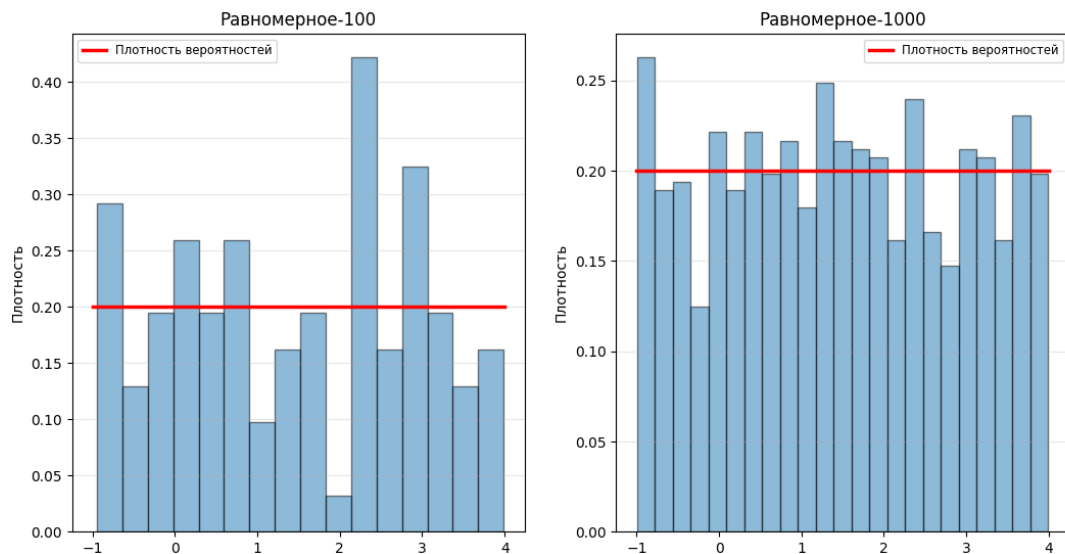
1  def f_x(x: float) -> float:
2      return 1 / (4 + 1)
3
4  fig, axes = plt.subplots(1, 2)
5
6  for col, size in enumerate([100, 1000]):
7      ax = axes[col]
8      sample = data["uniform"][size]["X"]
9
10     k = int(1 + 3.322 * np.log10(len(sample)))
11     h = (max(sample) - min(sample)) / k
12
13     endpoints = np.linspace(min(sample), max(sample), k + 1)
14     heights = []
15     for i in range(k):
16         left = endpoints[i]
17         right = endpoints[i+1]
18         count = ((sample >= left) & (sample < right)).sum()
19
20         density_height = count / (size * h)
```

```

21     heights.append(density_height)
22
23     bin_centers = endpoints[:-1] + h / 2
24     ax.bar(bin_centers, heights, width=h, alpha=0.5,
25            edgecolor='black')
26
27     k = np.linspace(-1, 4, 100)
28     ax.plot(k, [f_x(i) for i in k], 'r-', lw=2.5,
29            label='Плотность вероятностей')
30
31     ax.set_title(f"Равномерное-{size}")
32     ax.set_ylabel("Плотность")
33     ax.legend(fontsize='small')
34     ax.grid(axis='y', alpha=0.3)

```

Сами графики получились такими:



4.2 Распределение Бернулли

В случае распределения Бернулли случайная величина может принимать только 2 значения: 0 и 1. В связи с этим $k = 2$, то бишь будет частотный интервал для 0 и 1.

Функция вероятности распределения Бернулли имеет вид:

$$f_X(x) = \begin{cases} p, & \text{если } x = 1 \\ 1 - p, & \text{если } x = 0 \\ 0, & \text{иначе} \end{cases}$$

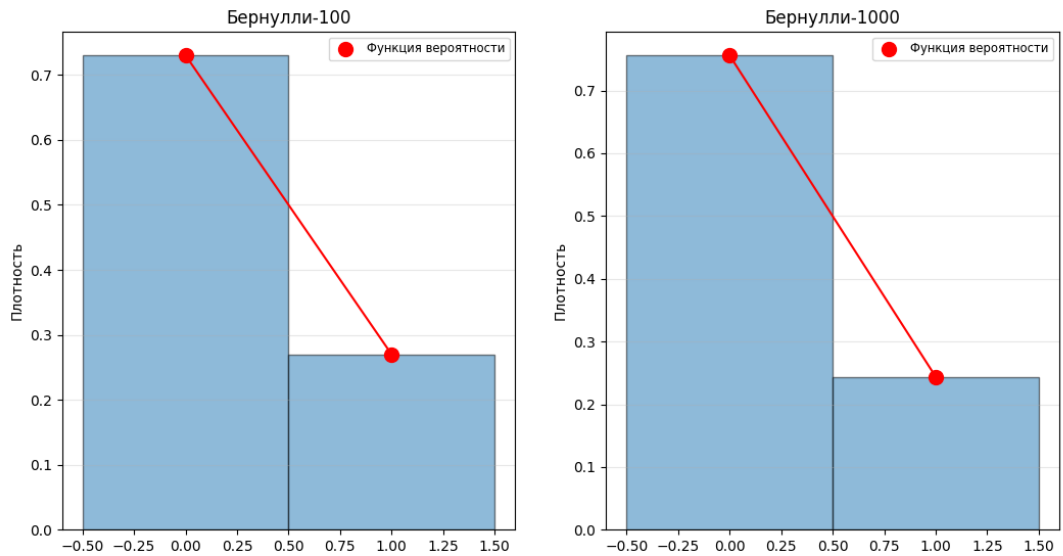
Таким образом код выглядит так:

```
1  fig, axes = plt.subplots(1, 2)
2
3  for col, size in enumerate([100, 1000]):
4      ax = axes[col]
5      sample = data["bernoulli"][size]["X"]
6
7      m_1 = sum(sample)
8      m_0 = size - m_1
9
10     w_0 = m_0 / size
11     w_1 = m_1 / size
12
13     heights = [w_0, w_1]
14
15     ax.bar([0, 1], heights, width=1, alpha=0.5,
16           edgecolor='black')
17
18     ax.scatter([0, 1], heights, s=100, zorder=5,
19               label='Функция вероятности', color="red")
20     ax.plot([0, 1], heights, color="red")
21
22     ax.set_title(f"Бернулли-{size}")
23     ax.set_ylabel("Плотность")
24     ax.legend(fontsize='small')
25     ax.grid(axis='y', alpha=0.3)
```

График будет выглядеть так:

4.3 Биномиальное распределение

В случае биномиального распределения интервал будет определяться целым числом от 0 до n . Функция вероятности распределения биномиального распре-



деления имеет вид:

$$f_X(x) = C_n^x p^x (1 - p)^{n-x}$$

Таким образом код будет иметь вид:

```

1  def combinations(n, k) -> float:
2      return math.factorial(n) / (math.factorial(k) *
3          math.factorial(n - k))
4
5  def f_x(x: float) -> float:
6      return combinations(17, x) * (0.25 ** x) * ((1 -
7          0.25)**(17 - x))
8
9  fig, axes = plt.subplots(1, 2)
10
11  for col, size in enumerate([100, 1000]):
12      ax = axes[col]
13      sample = data["binom"][size]["X"]
14
15      k = np.arange(0, 17 + 1)
16      counts = [np.sum(sample == i) for i in k]
17
18      heights = [m_i / size for m_i in counts]
19
20      ax.bar(k, heights, width=h, alpha=0.5, edgecolor='black')

```

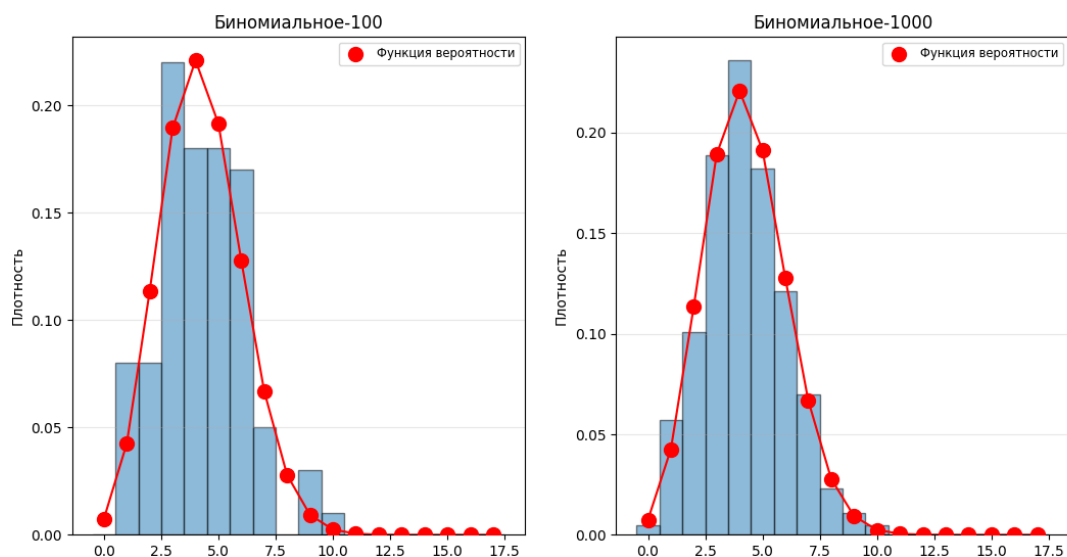


```

19
20 ax.scatter(k, [f_x(i) for i in k], s=100, zorder=5,
21            label='Функция вероятности', color="red")
22
23 ax.plot(k, [f_x(i) for i in k], color="red")
24
25 ax.set_title(f"Биномиальное-{size}")
26 ax.set_ylabel("Плотность")
27 ax.legend(fontsize='small')
28 ax.grid(axis='y', alpha=0.3)

```

Графики получились такими:



4.4 Нормальное распределение

Плотность вероятности для нормального распределения имеет вид: $f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-M)^2}{2\sigma^2}}$

Код аналогичен равномерному распределению:

```

1 def f_x(x: float) -> float:
2     return 1 / (5 * np.sqrt(2 * np.pi)) * np.exp(-(x - 15) **
3               2 / (2 * 5 ** 2))

```

```

4  fig, axes = plt.subplots(1, 2)
5
6  for col, size in enumerate([100, 1000]):
7      ax = axes[col]
8      sample = data["norm"][size]["X"]
9
10     k = int(1 + 3.322 * np.log10(len(sample)))
11     h = (max(sample) - min(sample)) / k
12
13     endpoints = np.linspace(min(sample), max(sample), k + 1)
14     heights = []
15     for i in range(k):
16         left = endpoints[i]
17         right = endpoints[i+1]
18         count = ((sample >= left) & (sample < right)).sum()
19
20         density_height = count / (size * h)
21         heights.append(density_height)
22
23     bin_centers = endpoints[:-1] + h / 2
24     ax.bar(bin_centers, heights, width=h, alpha=0.5,
25            edgecolor='black')
26
27     k = np.linspace(min(sample), max(sample), 100)
28     ax.plot(k, [f_x(i) for i in k], 'r-', lw=2.5,
29            label='Плотность вероятностей')
30
31     ax.set_title(f"Нормальное-{size}")
32     ax.set_ylabel("Плотность")
33     ax.legend(fontsize='small')
34     ax.grid(axis='y', alpha=0.3)

```

График вышел таким:

